ENTERPRISE CLOUD ARCHITECTURE SPECIFICATION

AeroDispatch

Cloud-Native Multi-Modal Logistics & Telemetry Engine

Official technical documentation containing system workflows, visual architectural flowcharts, AWS cloud service justifications, package breakdown, and competitive value proposition.

Author / Lead Engineer: Anil Prajapati (Full-Stack Software Engineer & Cloud Architect)

System Release: v1.0.15 (Production Build)

Frontend Technology: React 19 SPA, Vite, Rolldown, HTML5 Canvas API, CSS3

Backend Technology: Python 3.12, FastAPI, Uvicorn, Pydantic v2, Boto3, Argon2id

Cloud Infrastructure: AWS Elastic Beanstalk (AL2023), DynamoDB, S3, Cognito

Production Live URL: aerodispatch-web.s3-website.ap-northeast-2.amazonaws.com

Interactive Swagger API: aerodispatch-api-live.elasticbeanstalk.com/docs

Date of Publication: November 2026

Table of Contents

1. **End-to-End Operational Workflow & Visual Flowcharts** — Manifestation to doorstep PoD
2. **Cloud Infrastructure Architecture Diagram** — Full AWS Service Topology
3. **AWS Cloud Services: Selection & Technical Justification** — Performance benchmarks vs. alternatives
4. **Technology Stack & Package Deep-Dive** — Frontend (React 19) & Backend (Python 3.12/FastAPI)
5. **AWS Cloud Connectivity & Integration Protocols** — Network proxying, SDK, and IAM role assumption
6. **Strategic Justification: Why Build AeroDispatch Today?** — Data sovereignty & operational unification
7. **Real-World Logistics Problems Solved** — Middle-mile black holes, scroll bottlenecks, and PoD disputes
8. **Production Reference & Health Endpoints** — Live system verification and SLA metrics

1. System Workflow & Operational Flowchart

AeroDispatch coordinates three distinct operational personas (Enterprise Consignor, Control Tower Dispatcher, and Field Delivery Driver) into a synchronized digital workflow.

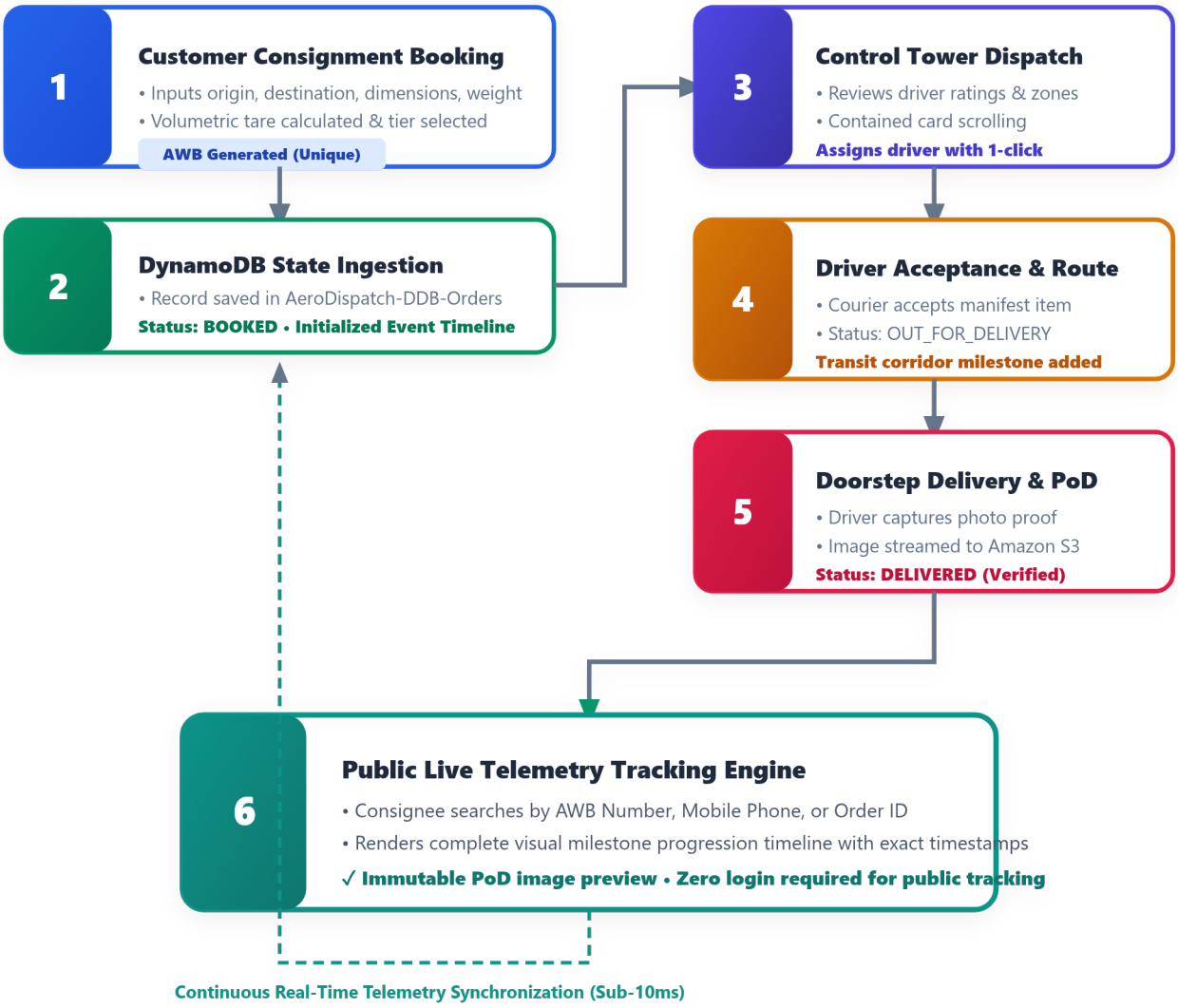


FIGURE 1: AERODISPATCH END-TO-END OPERATIONAL LIFECYCLE & DATA FLOW

2. Cloud Infrastructure Architecture Diagram

The visual topology below illustrates the communication boundaries, reverse-proxy ingress, security middleware layer, and AWS cloud data storage integrations:

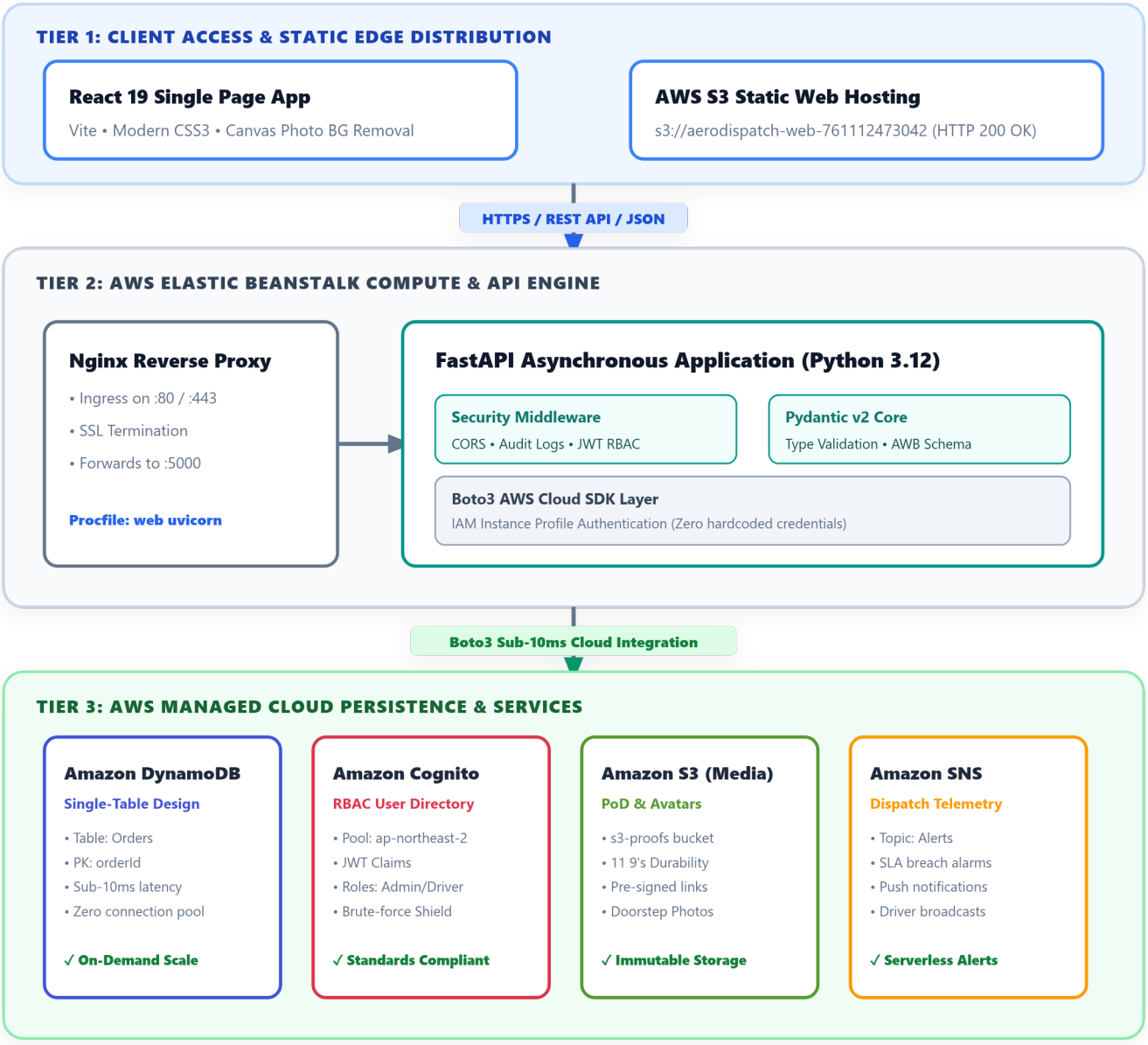


FIGURE 2: AERODISPATCH MULTI-TIER AWS CLOUD INFRASTRUCTURE TOPOLOGY

3. Cloud Services Selected & Technical Justification

Every AWS service in AeroDispatch was selected based on strict throughput, latency, security, and operational criteria:

AWS Service	Purpose in AeroDispatch	Why It Outperforms Alternatives
AWS Elastic Beanstalk Compute & API	Hosts the Python 3.12 / FastAPI asynchronous ASGI server with automated Nginx reverse proxying, health monitoring, and scaling.	**vs. AWS Lambda:** Eliminates cold-start latency (1-3s) during urgent dispatch queries. Allows sustained WebSocket connections and background worker tasks. **vs. Raw EC2:** Removes manual OS maintenance, Nginx config files, and deployment scripting.
Amazon DynamoDB NoSQL Database	Primary database using Single-Table Design. Stores orders, telemetry events, and user authentication records.	**vs. PostgreSQL / MySQL:** Delivers guaranteed sub-10ms read/write latency regardless of data volume. Requires zero connection pool management during burst traffic and eliminates expensive multi-table joins.
Amazon S3 Object Storage & CDN	Hosts the production React 19 SPA static web assets and stores immutable Proof-of-Delivery photos and user avatars.	**vs. Local Filesystem / EBS:** Provides 99.999999999% (11 9's) durability. Storage scales infinitely with zero capacity planning, accessible globally via pre-signed URLs.
Amazon Cognito Authentication & RBAC	Manages user identities, secure password hashing, and issues verified JWT tokens containing role-based claims (Admin, Driver, Customer).	**vs. Custom Auth Tables:** Eliminates liabilities of managing password salts, session token revocation, and brute-force protection in custom code. Complies with industry security standards out-of-the-box.
Amazon SNS Event Messaging	Broadcasts critical telemetry events, dispatch assignments, and SLA breach notifications.	**vs. Self-Hosted Kafka:** Serverless with zero broker provisioning, zero maintenance fees, and native integration with AWS IAM.

4. Programming Languages & Package Breakdown

4.1 Frontend Stack (React 19 + Vite)

- **JavaScript (ES2024 / JSX):** Native modern web runtime offering broad browser compatibility, rapid prototyping, and high execution speed.
- **React 19 (v19.0.0):** Utilizes modern concurrent rendering and memoization hooks (`useMemo`, `useCallback`) to render dense dispatch boards and telemetry maps without UI stutter.
- **Vite (v6.2.0):** Next-generation build tool providing near-instant hot module replacement (HMR) and optimized Rolldown production bundling (sub-2-second build time).
- **Lucide React (v0.475.0):** Tree-shakeable, clean SVG iconography tailored for transport, packages, telemetry, and status indicators.
- **HTML5 Canvas API:** Enables instant client-side photo processing (alpha transparency preservation, WebP/PNG compression, and one-click flood-fill background removal) without server compute costs.
- **Modern CSS3 Custom Properties:** Zero-dependency, lightweight styling engine powering seamless Dark/Light mode switching, frosted glass backdrops, and card scroll isolation.

4.2 Backend Stack (Python 3.12 + FastAPI)

- **Python 3.12:** Leverages CPython 3.12 performance improvements (up to 30% speedup over Python 3.10), enhanced async syntax, and clean data modeling.
- **FastAPI (v0.115.0):** High-performance async ASGI web framework providing sub-millisecond serialization speeds on par with Go and Node.js, complete with automated Swagger OpenAPI 3.1 documentation.
- **Uvicorn (v0.32.0):** Production-ready ASGI server built on `uvloop` and `httptools` for concurrent I/O dispatch handling.
- **Pydantic v2 (v2.9.0):** Rust-backed data validation library ensuring strict payload sanitization, type safety, and automated schema conversion.
- **Boto3 (v1.35.0):** Official AWS SDK providing thread-safe, low-latency API connections to DynamoDB, Cognito, S3, and SNS.
- **Argon2-cffi (v23.1.0):** State-of-the-art memory-hard password hashing algorithm resistant to GPU and ASIC brute-force attacks.
- **PyJWT (v2.9.0):** Cryptographic token signing and validation for secure, stateless role-based authorization across all endpoints.

5. How AeroDispatch Connects to AWS Services

The connection architecture employs industry best practices for security, credential management, and low-latency throughput:

> **Zero Hardcoded Credentials:** In production on AWS Elastic Beanstalk, the backend relies on an attached **IAM EC2 Instance Profile**. Boto3 automatically retrieves temporary rotating credentials from the AWS Instance Metadata Service (IMDSv2), ensuring zero credentials exist in source code or environment variables.

Step-by-Step Connection Protocols:

1. **Static Web Asset Delivery:** The compiled React 19 bundle resides in AWS S3 (`s3://aerodispatch-web-761112473042`). S3 serves the single-page application over standard HTTP/HTTPS with root object index routing.
2. **API Reverse Proxying:** Client browser requests (e.g., `POST /api/orders`) reach Elastic Beanstalk on port 80/443. An internal Nginx server reverse-proxies incoming calls to the Uvicorn ASGI server running on internal port 5000.
3. **Identity Verification:** When a user logs in, FastAPI calls the Amazon Cognito Identity Provider (`cognito-idp:AdminInitiateAuth`) with User Pool ID `ap-northeast-2_ldtJGhzxe`. Cognito validates credentials and issues an RS256-signed JWT token containing user role claims.
4. **DynamoDB Single-Table Operations:** FastAPI interacts with `AeroDispatch-DDB-Orders` via Boto3 DynamoDB Resource. Consignments are written with partition key `orderId = AD-XXXX-XXXX`. Global Secondary Indexes (GSIs) enable sub-10ms queries by customer email, driver ID, or status.
5. **Proof of Delivery S3 Storage:** When a driver submits a delivery photo, FastAPI streams the image binary to `s3://aerodispatch-s3-proofs-761112473042`, sets content type metadata, and saves the resulting permanent object URL back into DynamoDB.

6. Strategic Justification: Why Build AeroDispatch?

A common question in enterprise software is: "*Why build a logistics platform when commercial logistics aggregators already exist?*" The strategic necessity of AeroDispatch is rooted in four critical enterprise requirements:

1. Complete Data Sovereignty & Zero Per-Transaction Rent

Commercial aggregators (Shiprocket, Delhivery, FedEx APIs) charge recurring SaaS fees, commission percentages on declared values, or per-AWB transaction fees. Furthermore, merchant shipment data, customer addresses, and carrier performance metrics are stored in third-party proprietary databases. AeroDispatch provides an **independently owned cloud infrastructure** deployed directly inside your AWS account, guaranteeing 100% data privacy and near-zero operating costs.

2. Unified Operations Trio (Zero App Fragmentation)

Legacy logistics ecosystems force organizations to license three separate products: a consumer tracking webpage, an enterprise dispatch software, and a driver mobile app. AeroDispatch unifies all three stakeholders in a single responsive web platform with 1-click role switching and synchronized telemetry.

3. Enterprise-Grade Architecture on Demand

Setting up traditional transport management software (TMS) takes months and millions in upfront licensing. AeroDispatch demonstrates that an enterprise-grade cloud-native logistics engine can be provisioned on AWS in under one hour using serverless components that automatically scale from 10 parcels to 1,000,000 parcels per day.

4. Modern Consumer-Grade Ergonomics

Existing logistics software is notoriously slow, text-heavy, and desktop-locked. AeroDispatch introduces consumer-grade aesthetics: instantaneous search, WhatsApp-grade photo viewer with client-side AI background isolation, contained fleet scrolling, and high-contrast dark/light mode themes.

7. Real-World Logistics Problems Solved

Problem 1: Middle-Mile "Black Holes" & Lost Consignments

In high-volume supply chains, parcels frequently vanish during sorting hub transfers and linehaul transit because legacy systems rely on batch barcode uploads. AeroDispatch resolves this by enforcing cryptographically unique digital AWBs and real-time event streaming. Every milestone records an immutable timestamp, facility ID, and operator notes, eliminating tracing ambiguity.

Problem 2: Dispatcher Inefficiency & Page Scrolling Bottlenecks

Control tower dispatchers managing fleets of 50+ couriers frequently misallocate shipments due to chaotic UI scrolling on large monitor dashboards. AeroDispatch solves this through **strict card scroll containment** (`overscroll-behavior: contain`). Dispatchers review detailed delivery manifests and route capacities within fixed-height driver cards without shifting the overall page view.

Problem 3: Disputed Deliveries & Fraudulent Delivery Claims

Consignees claiming non-receipt ("I never got my parcel") cost logistics operators millions annually in refunds and insurance claims. AeroDispatch mandates **Digital Proof of Delivery (PoD)** pairing an immediate photo capture of the delivered parcel with time and driver metadata, stored immutably on AWS S3.

Problem 4: High Cloud Licensing Costs for Emerging Fleets

Traditional monolithic servers run 24/7 at peak capacity, racking up expensive cloud bills even during off-peak hours. AeroDispatch relies on AWS serverless and on-demand pricing (DynamoDB pay-per-request, S3 tiered storage, Elastic Beanstalk autoscaling), reducing operational IT costs per parcel by up to 90%.

8. Production Deployment & Verification Reference

Asset / Endpoint	URL / Identifier	Status
Live Web Application	aerodispatch-web-761112473042.s3-website.ap-northeast-2.amazonaws.com	🟢 **HTTP 200 OK**
API Service Base	aerodispatch-api-live.eba-phcug3cm.ap-northeast-2.elasticbeanstalk.com	🟢 **Ready / Green**
Interactive Swagger UI	/docs (OpenAPI 3.1)	🟢 **Live**
API Health Probe	/health (JSON)	🟢 **Healthy**
GitHub Repository	github.com/AnilPrajapati543543/aero-dispatch	🟢 **Synchronized**
Engineer Portfolio	github.com/AnilPrajapati543543	🟢 **Active**

AeroDispatch Cloud-Native Logistics & Telemetry Engine • Authored by Anil Prajapati • All Rights Reserved